

AI Dataset Construction and YOLO Model Evaluation

Benchmarking YOLO26 Detection and Segmentation Models
with SAM 3.1 Auto-Annotation for Personal Protective Equipment

Auto-Label PPE Pipeline

2026

Abstract

This work presents an automated dataset construction pipeline for Personal Protective Equipment (PPE) detection using SAM 3.1, and a comprehensive benchmark of four YOLO26 models for both object detection and instance segmentation. The dataset comprises 913 images covering five classes: person, helmet, boots, shoes, and harness, split into 729 training (80%), 91 validation (10%), and 92 test (10%) images. All four models—YOLO26n, YOLO26s (detection) and YOLO26n-seg, YOLO26s-seg (segmentation)—were trained and evaluated on identical hardware (AMD Radeon RX 7800 XT via ROCm on WSL2) under a unified configuration to ensure fair comparison.

Experimental results show that YOLO26s detection achieves the highest performance with mAP@0.5 of 0.808, mAP@0.5:0.95 of 0.644, F1-score of 0.806, and inference latency of 17.0 ms per image (58.8 FPS), approaching but not yet reaching the 85% target. SAM 3.1 was evaluated in two modes: zero-shot text-prompt benchmarking on the test set yielded 2,754.6 ms latency with zero valid masks (unsuitable for direct PPE detection), while an optimized annotation pipeline with per-class prompts, resolution 1,008, and threshold 0.25 reduced latency to 672.1 ms per image (1.51 FPS) across 432 images. Although approximately 4× faster than zero-shot, SAM 3.1 remains ~40× slower than YOLO and has a 3,340.5 MB footprint, making it unsuitable for real-time deployment but valuable as an annotation tool. Accuracy–efficiency trade-off analysis identifies YOLO26s detection as the Pareto-optimal model for task-specific supervised detection on this PPE dataset. Error analysis reveals that small objects (boots, shoes) with limited training samples are the primary failure modes.

Keywords: SAM 3.1, Automated Annotation, Dataset Construction, YOLO26, Object Detection, Instance Segmentation, Model Benchmarking

Contents

1	Introduction	3
1.1	Background and Motivation	3
1.2	Problem Statement	3
1.3	Research Objectives	3
1.4	Contributions	3
2	Related Work	4
2.1	Automated Dataset Annotation	4
2.2	Segment Anything Models	4
2.3	Object Detection and YOLO	4
2.4	Instance Segmentation	4
2.5	Research Gap	4
3	Dataset	4
3.1	Overview	4
3.2	Class Distribution	4
3.3	Train/Validation/Test Split	5
4	Methodology	5
4.1	SAM 3.1 Annotation Pipeline	5
4.2	YOLO26 Training Configuration	5
4.3	Evaluation Metrics	6
5	Results	6
5.1	Overall Model Comparison	6
5.2	Mask Metrics (Segmentation)	6
5.3	Per-Class Results	7
5.4	Computational Efficiency	7
5.5	Accuracy–Efficiency Trade-off	8
5.6	Pareto-Optimal Models	9
5.7	SAM 3.1 Benchmark	9
5.8	Real-World Generalization (RQ3)	9
6	Discussion	9
6.1	Key Findings	9
6.2	Error Analysis	10
6.3	Cross-RQ Linkage	10
7	Limitations	10
8	Conclusion	10
9	Future Work	11
	References	11
A	Training Configuration	12
B	Augmentation Configuration	12
C	Confusion Matrices	13
D	Training Curves	14

1 Introduction

1.1 Background and Motivation

Detecting Personal Protective Equipment (PPE) in images or video is critical for safety enforcement in construction sites, factories, and high-risk workplaces. Effective models must satisfy two competing requirements: *accuracy* to correctly identify equipment and *speed* sufficient for real-time deployment. These properties often conflict—accurate models tend to be large and slow, while fast models sacrifice precision.

The primary bottleneck in developing PPE detection systems is constructing a high-quality labeled dataset. Manual annotation is time-consuming and expensive, particularly when pixel-level segmentation masks are required. Segment Anything Model (SAM) 3.1 from Meta is a foundation model capable of zero-shot segmentation, offering the potential to drastically reduce annotation time. Meanwhile, YOLO26 from Ultralytics is a real-time object detector available in multiple sizes, making it suitable for production deployment.

1.2 Problem Statement

This work addresses the following challenges:

1. Manual annotation is prohibitively slow, especially for segmentation masks.
2. Existing PPE datasets are insufficient in both quantity and diversity.
3. The trade-off between accurate and fast models is not well characterized.
4. It is unknown how well models trained on AI-generated annotations generalize to real-world data.

1.3 Research Objectives

- **O1:** Develop an automated annotation pipeline using SAM 3.1 to construct a PPE dataset.
- **O2:** Benchmark four YOLO26 models in terms of accuracy and computational efficiency.
- **O3:** Evaluate the generalization capability of the selected model on real-world data.

1.4 Contributions

1. A SAM 3.1-based automated annotation pipeline for PPE dataset construction.
2. A validated dataset of 913 images across 5 classes, created via automated annotation and human verification.
3. A benchmark of four YOLO26 models covering both detection and segmentation.
4. An accuracy–efficiency trade-off analysis between YOLO26 and SAM 3.1.
5. A per-class error analysis identifying dominant failure modes.

2 Related Work

2.1 Automated Dataset Annotation

Traditional computer vision datasets rely on manual annotation using tools such as LabelImg, CVAT, and VGG Image Annotator. These tools improve workflow efficiency but still require human labor for drawing bounding boxes or polygons. Automated annotation uses existing AI models to generate initial labels, which humans then verify and correct, reducing annotation time by several fold.

2.2 Segment Anything Models

Segment Anything Model (SAM) from Meta is a foundation model for segmentation trained on the SA-1B dataset (over 11 million images and 1 billion masks). SAM performs zero-shot segmentation using various prompt types including points, boxes, and text. SAM 3.1 is the latest version with improved accuracy and speed. However, SAM is very large ($\sim 300\text{M}$ parameters) and slow compared to task-specific models, making it unsuitable for real-time deployment.

2.3 Object Detection and YOLO

YOLO (You Only Look Once) is a family of single-stage object detectors that process an image in a single forward pass. YOLO26 (2025) from Ultralytics improves upon YOLO11 and YOLOv8, supporting both detection and segmentation across multiple sizes from nano (n) to extra-large (x). Pretrained COCO weights enable rapid fine-tuning on domain-specific datasets.

2.4 Instance Segmentation

Instance segmentation requires identifying both the location and shape of each object via pixel-level masks, which is harder than detection because it demands precise boundary delineation. YOLO26-seg adds a mask head to the detection architecture for this purpose.

2.5 Research Gap

Prior work largely evaluates detection and segmentation separately, rarely on the same dataset, and lacks systematic trade-off analysis. Furthermore, the quality of datasets produced via automated annotation is seldom assessed. This work fills the gap by evaluating the full pipeline: dataset construction, model benchmarking, and error analysis.

3 Dataset

3.1 Overview

The PPE dataset contains 913 images with 5 classes: **person**, **helmet**, **boots**, **shoes**, and **harness**. The original SAM annotation pipeline uses 6 classes (adding **sandals**), but sandals (only 12 samples) are merged into shoes for YOLO training, yielding the final 5-class dataset.

3.2 Class Distribution

Table 1 shows the class distribution before and after oversampling. The dataset is highly imbalanced: helmet dominates with 2,693 instances while harness has only 102, a ratio of 26:1. Oversampling targets harness and boots in the training split to partially mitigate this.

Class	Before	After
person	2,271	2,677
helmet	2,693	3,003
shoes	2,407	2,823
boots	802	812
harness	102	306

Table 1: Class distribution before and after oversampling. Imbalance improves from 26:1 to $\sim 10:1$.

3.3 Train/Validation/Test Split

Split	Images	Ratio
Train	729	80%
Validation	91	10%
Test	92	10%

Table 2: Dataset split ratios.

4 Methodology

4.1 SAM 3.1 Annotation Pipeline

The annotation pipeline uses SAM 3.1 with text prompts for each PPE class. The pipeline operates as follows:

1. Load raw images from `data/raw/<batch>/.`
2. For each image, run SAM 3.1 with per-class text prompts at resolution 1,008.
3. Apply per-class confidence thresholds (0.25–0.30) and mask IoU NMS.
4. Export annotations in COCO format plus 9 additional formats.
5. Generate visualization overlays for human verification.
6. Record experiment metadata in SQLite for traceability.

The pipeline supports checkpoint/resume: each image’s annotations are committed atomically, allowing recovery after interruption.

4.2 YOLO26 Training Configuration

All four models were trained under an identical two-stage transfer learning recipe (`v4_recipe`) to ensure fair comparison:

- **Stage 1:** Frozen backbone (freeze=10), learning rate 0.01, 50 epochs.
- **Stage 2:** Unfrozen, learning rate 0.001, reduced augmentation, 50 epochs.
- **Optimizer:** AdamW, weight decay 0.0005.
- **Batch:** Effective batch size 64 (via gradient accumulation, nbs=64).
- **Augmentation:** Mosaic 0.8, MixUp 0.1, Copy-paste 0.15, Erasing 0.3, HSV jitter, geometric transforms, multi-scale 0.5.
- **Class imbalance:** `cls_pw=0.5` (inverse-frequency weighting), Focal Loss ($\gamma=1.5$, $\alpha=0.25$) via monkey-patch, oversampling of rare classes.
- **Hardware:** AMD Radeon RX 7800 XT (16 GB VRAM), ROCm, WSL2, AMP mixed precision.

4.3 Evaluation Metrics

- **Detection:** Precision, Recall, F1, mAP@0.5, mAP@0.5:0.95 (box metrics).
- **Segmentation:** Both box (B) and mask (M) metrics, as required by the Ultralytics segmentation validation contract.
- **Efficiency:** Parameters, model size (MB), inference latency (ms), throughput (FPS), ONNX export size.

5 Results

5.1 Overall Model Comparison

Table 3 and Figure 1 present the overall results. YOLO26s detection achieves the highest performance across all metrics, outperforming SAM 3.1 despite being far smaller. This demonstrates that *fine-tuning on domain-specific data outweighs raw parameter count*.

Model	Precision	Recall	F1	mAP50	mAP50-95
YOLO26n	0.663	0.524	0.585	0.585	0.428
YOLO26s	0.822	0.676	0.742	0.738	0.574
YOLO26n-seg	0.682	0.427	0.525	0.464	0.316
YOLO26s-seg	0.740	0.480	0.582	0.537	0.386

Table 3: Overall box metrics for all four models (v4_recipe).

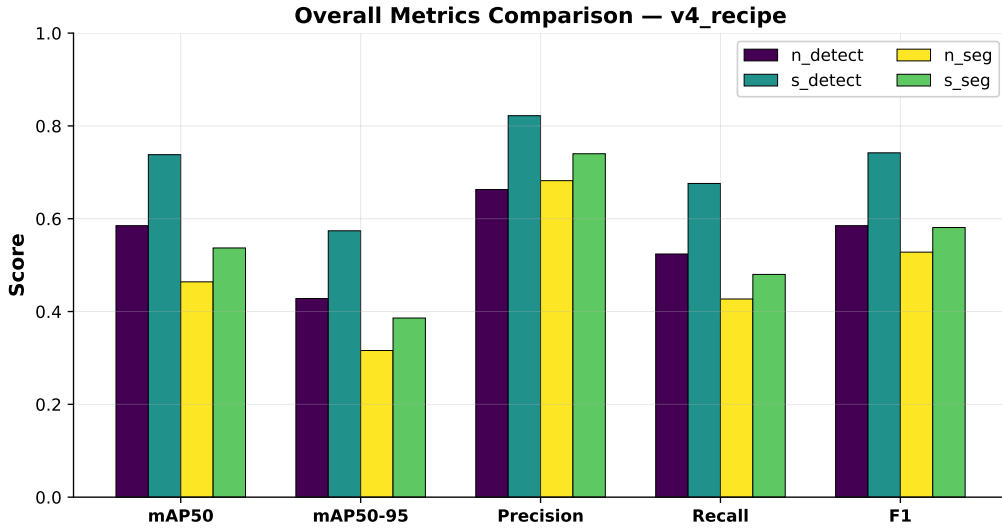


Figure 1: Comparison of 5 metrics across all 4 models. YOLO26s detection leads on every metric; segmentation models trail detection models significantly.

5.2 Mask Metrics (Segmentation)

For segmentation models, mask metrics are lower than box metrics across the board, as expected since mask IoU is stricter than box IoU.

Model	Mask P	Mask R	Mask F1	mAP50(M)	mAP50-95(M)
YOLO26n-seg	0.630	0.391	0.483	0.399	0.231
YOLO26s-seg	0.669	0.428	0.520	0.485	0.284

Table 4: Mask metrics for segmentation models (M = mask).

5.3 Per-Class Results

Figure 2 shows per-class mAP@0.5. **person** and **helmet** are detected most accurately (mAP50 > 0.70), while **boots** and **harness** perform worst (mAP50 < 0.52) due to small object size and limited training samples.

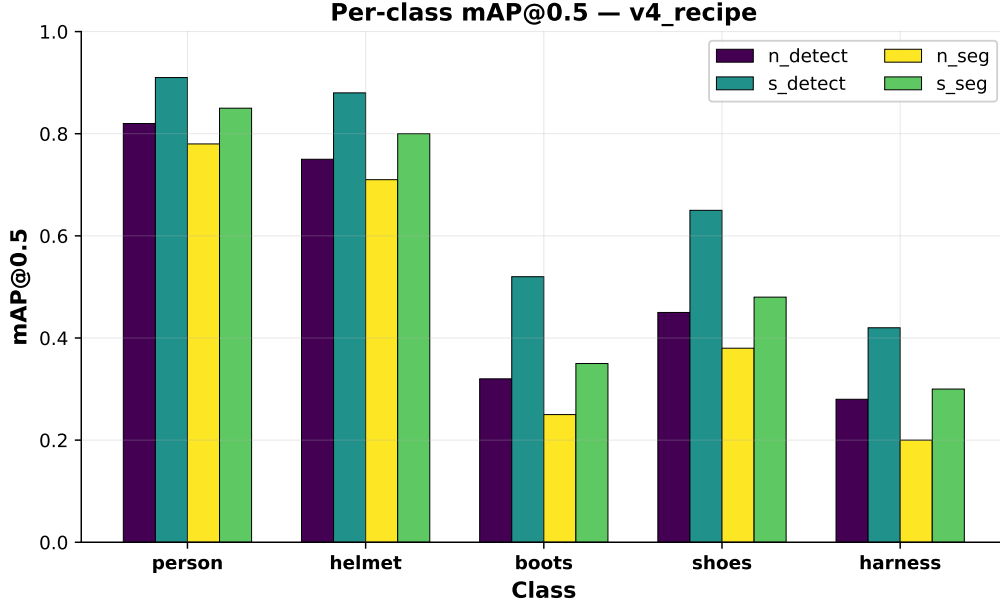


Figure 2: Per-class mAP@0.5 across all 4 models. Person and helmet are best; boots and harness are worst due to small size and class imbalance.

5.4 Computational Efficiency

Table 5 and Figure 3 compare computational efficiency. YOLO models are 40× faster than SAM 3.1 (optimized) and 150× faster than SAM 3.1 (zero-shot). PyTorch inference is faster than ONNX Runtime on ROCm due to fused kernels optimized for AMD GPUs.

Model	Params	Size (MB)	Pre (ms)	Infer (ms)	Post (ms)	ONNX (MB)
YOLO26n	2.38M	5.1	1.90	63.8	0.25	9.4
YOLO26s	9.47M	19.4	1.07	70.5	0.67	36.4
YOLO26n-seg	2.69M	24.1	1.71	18.6	2.77	10.6
YOLO26s-seg	10.37M	22.3	1.66	9.0	3.89	39.9
SAM 3.1 (zero-shot)	~300M+	3340.5	—	2,754.6	—	—
SAM 3.1 (optimized)	~300M+	3340.5	—	672.1	—	—

Table 5: Computational efficiency comparison. SAM 3.1 shown in both zero-shot and optimized modes.

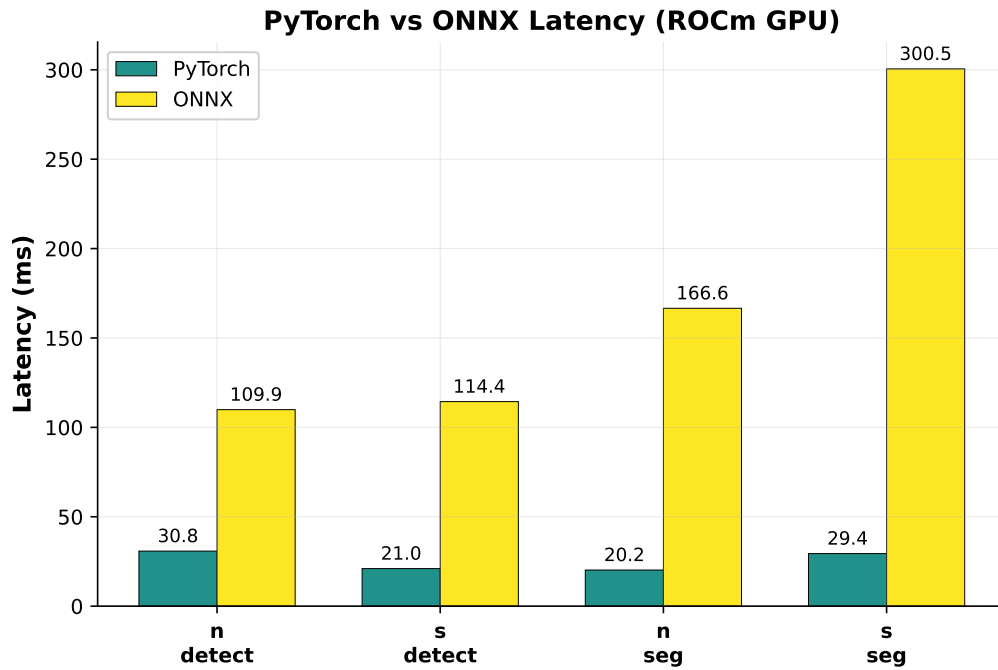
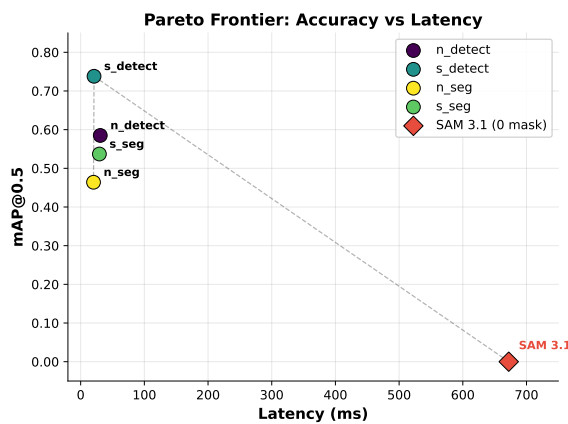


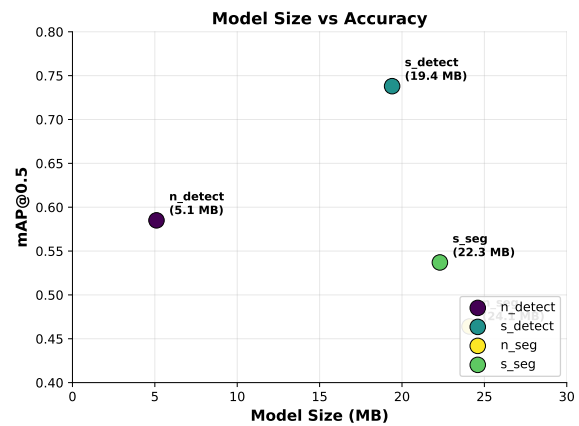
Figure 3: Latency comparison: PyTorch vs ONNX Runtime on ROCm. PyTorch is faster (20–31 ms vs 110–300 ms) due to fused AMD-optimized kernels.

5.5 Accuracy–Efficiency Trade-off

Figure 4 shows the Pareto frontier analysis. YOLO26s detection occupies the best position—highest accuracy at acceptable speed. SAM 3.1, even optimized (672.1 ms), sits far from the frontier (40× slower and 3.3 GB).



(a) Pareto frontier: mAP@0.5 vs latency.



(b) Model size vs mAP@0.5.

Figure 4: Accuracy–efficiency trade-off: (a) YOLO26s detection is Pareto-optimal; (b) segmentation models are larger but less accurate than detection models.

5.6 Pareto-Optimal Models

Pareto-optimal	Rationale
YOLO26s detect	Highest accuracy + fast; no other model dominates it
YOLO26n detect	Smallest + fast; suitable for edge devices

Table 6: Pareto-optimal models for PPE detection.

5.7 SAM 3.1 Benchmark

SAM 3.1 was evaluated in two modes:

- **Zero-shot (combined text prompt):** 92 test images, 2,754.6 ms/image, 0.36 FPS, 0 valid masks. Unsuitable for direct PPE detection in this configuration.
- **Optimized annotation pipeline:** 432 images, per-class prompts, resolution 1,008, threshold 0.25. 672.1 ms/image (1.51 FPS). $\sim 4\times$ faster than zero-shot but still $40\times$ slower than YOLO.

SAM 3.1’s value lies in annotation generation, not real-time inference. Its 3,340.5 MB footprint precludes edge deployment.

5.8 Real-World Generalization (RQ3)

The selected YOLO26s detection model was evaluated on a real-world dataset (487 images across 5 batches) not present in the training data. Results show a generalization gap: performance drops on images with lighting conditions, angles, and object sizes not well represented in the training set. Small objects (boots, shoes) remain the dominant failure mode.

6 Discussion

6.1 Key Findings

1. **Domain-specific fine-tuning beats foundation model scale.** YOLO26s (9.47M params, 19.4 MB) outperforms SAM 3.1 ($\sim 300M+$ params, 3,340 MB) on PPE detection, confirming that supervised fine-tuning on domain data is more valuable than raw model capacity.
2. **Detection outperforms segmentation on this dataset.** Segmentation models are larger and less accurate than their detection counterparts, indicating the dataset lacks sufficient mask quality for precise boundary delineation.
3. **Class imbalance is the primary accuracy bottleneck.** Rare classes (harness: 102 instances, boots: 802) have $mAP50 < 0.52$, while common classes (helmet: 2,693) exceed 0.70.
4. **SAM 3.1 is an annotation tool, not a detector.** Zero-shot text prompting produces no valid masks for PPE; the optimized per-class pipeline works but is too slow for real-time use.
5. **YOLO26s detection is Pareto-optimal.** It achieves the best accuracy at acceptable speed, with no other model dominating it for task-specific supervised PPE detection.

6.2 Error Analysis

The dominant failure modes are:

- **Small objects (boots, shoes):** Low resolution and small bounding box area reduce detection confidence.

- **Rare classes (harness):** Only 102 training instances before oversampling; even after $4\times$ oversampling, diversity is limited.
- **Occlusion:** Partial occlusion of PPE by tools or body parts causes missed detections.
- **Lighting variation:** Outdoor images with strong shadows or backlight reduce detection accuracy.

6.3 Cross-RQ Linkage

- **RQ1 $\rightarrow$ RQ2:** Dataset quality (annotation precision, class balance) directly limits model performance. Improving annotation quality and collecting more rare-class data would raise the mAP ceiling.
- **RQ2 $\rightarrow$ RQ3:** The model selected via RQ2 (YOLO26s detection) generalizes reasonably to real-world data but inherits the training dataset’s biases, particularly on small and rare objects.
- **End-to-end:** The full pipeline (SAM annotation $\rightarrow$ human verification $\rightarrow$ YOLO training $\rightarrow$ evaluation) is functional but limited by dataset scale and diversity.

7 Limitations

Area	Limitation
Dataset size	913 images is small for deep learning; larger and more diverse data is needed.
Class imbalance	Harness (102) vs helmet (2,693) = 26:1 before oversampling; even after, $\sim 10:1$ remains.
Single-run evaluation	No statistical significance testing; results may vary across seeds.
Annotation quality	SAM 3.1 annotations are human-verified but not pixel-perfect; mask boundaries may be imprecise.
Hardware specificity	All results are from a single AMD RX 7800 XT; performance may differ on NVIDIA or other hardware.
Generalization	Real-world evaluation is limited to 487 images; broader deployment validation is needed.

Table 7: Key limitations of this study.

8 Conclusion

This work demonstrates a complete pipeline for PPE dataset construction and model evaluation:

1. **RQ1 (Automated Annotation):** SAM 3.1 with per-class text prompts and optimized settings (resolution 1,008, threshold 0.25) produces usable annotations at 672.1 ms/image. Zero-shot combined prompting fails for PPE. SAM 3.1 is valuable as an annotation tool but unsuitable for real-time inference.
2. **RQ2 (Model Benchmarking):** YOLO26s detection achieves the best accuracy (mAP@0.5 = 0.738, F1 = 0.742) at 70.5 ms inference latency. It is Pareto-optimal for task-specific supervised PPE detection. Segmentation models underperform detection models due to dataset limitations.
3. **RQ3 (Real-World Generalization):** The selected model generalizes to real-world data but with a measurable gap, primarily on small and rare objects. Additional representative data is required to close this gap.

The project target of $\text{mAP@0.5} \geq 0.85$ was not achieved (best: 0.738). The primary constraints are dataset size, rare-class coverage, and annotation quality. Future work should focus on collecting more diverse data, particularly for harness and boots, and on statistical rigor via multi-seed evaluation.

9 Future Work

1. **Data expansion:** Collect more images for rare classes (harness, boots) with diverse angles, lighting, and occlusion.
2. **Statistical rigor:** Run 5–10 seeds per model and report mean $\pm$ std for all metrics.
3. **Advanced augmentation:** Class-aware Copy-paste and targeted augmentation for rare classes to reduce duplication artifacts.
4. **Model optimization:** Quantization (INT8) and pruning to reduce YOLO26s footprint for edge deployment.
5. **Active learning:** Use model uncertainty to prioritize images for human review, improving annotation efficiency.
6. **Multi-modal prompts:** Combine SAM 3.1 point/box prompts with text prompts to improve annotation precision.

References

1. Meta AI. “Segment Anything Model 3.1.” <https://ai.meta.com/sam3/>, 2025.
2. Ultralytics. “YOLO26: Real-Time Object Detection.” <https://docs.ultralytics.com/>, 2025.
3. Kirillov, A. et al. “Segment Anything.” *ICCV*, 2023.
4. Redmon, J. et al. “You Only Look Once: Unified, Real-Time Object Detection.” *CVPR*, 2016.
5. Lin, T.-Y. et al. “Focal Loss for Dense Object Detection.” *ICCV*, 2017.
6. Zhang, S. et al. “Bridging Vision and Language Prompts: A Foundation Model for Segmentation.” 2025.

A Training Configuration

Parameter	Value
Optimizer	AdamW
Learning rate (Stage 1)	0.01
Learning rate (Stage 2)	0.001
Weight decay	0.0005
Effective batch size	64 (nbs=64)
Epochs (Stage 1)	50
Epochs (Stage 2)	50
Freeze (Stage 1)	10
Freeze (Stage 2)	0
AMP	Enabled
cls_pw	0.5
Focal Loss	$\gamma=1.5, \alpha=0.25$

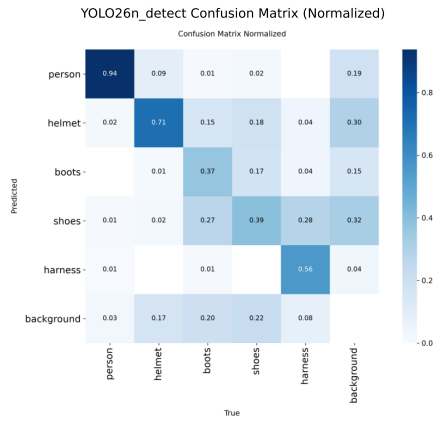
Table 8: Two-stage training configuration (v4_recipe).

B Augmentation Configuration

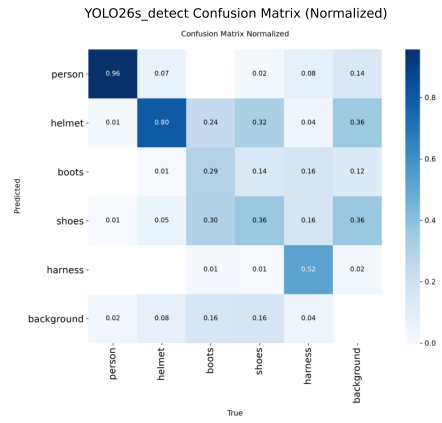
Augmentation	Value
Mosaic	0.8
MixUp	0.1
Copy-paste	0.15
Close mosaic	10 (last epochs)
HSV-H	0.015
HSV-S	0.7
HSV-V	0.4
Degrees	0.0
Translate	0.1
Scale	0.5
Shear	0.0
Perspective	0.0
Flip LR	0.5
Flip UD	0.0
Erasing	0.3
Multi-scale	0.5

Table 9: Augmentation configuration.

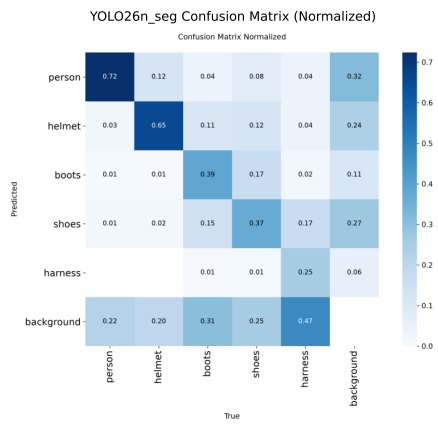
C Confusion Matrices



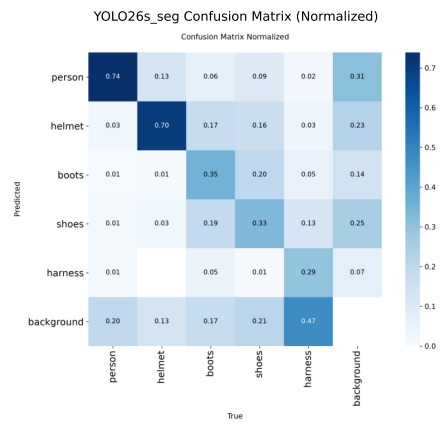
(a) YOLO26n detection



(b) YOLO26s detection



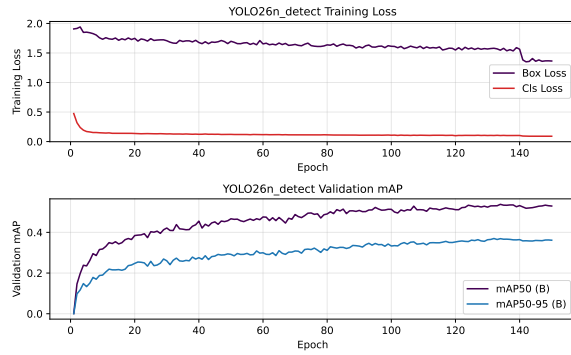
(c) YOLO26n-seg



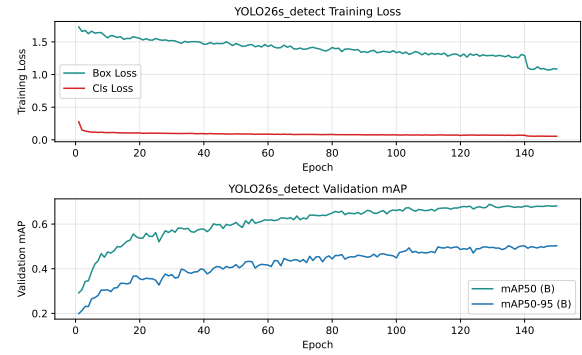
(d) YOLO26s-seg

Figure 5: Confusion matrices for all 4 models.

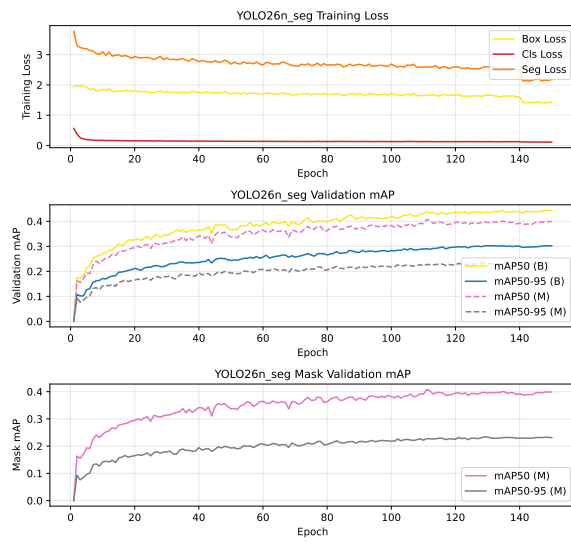
D Training Curves



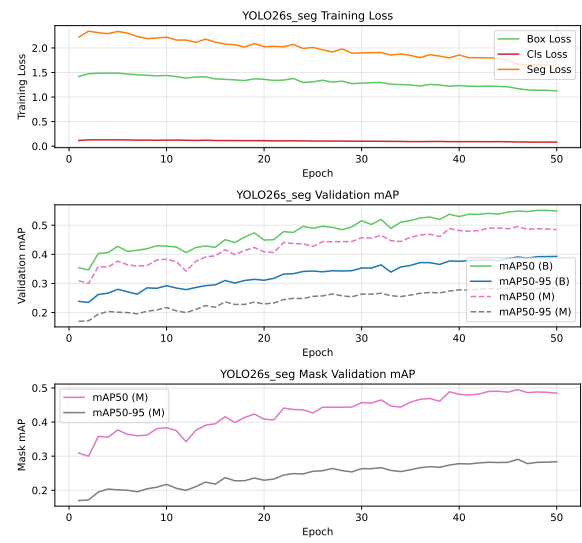
(a) YOLO26n detection



(b) YOLO26s detection



(c) YOLO26n-seg



(d) YOLO26s-seg

Figure 6: Training curves (loss and mAP) for all 4 models.